

INTERVIEW PREP NOTES

Java Programs & DSA Question Bank

✓ 26 core Java programs with full working code

✓ 60 top DSA questions, sorted Easy → Hard

✓ Real company tags, so you know what to expect

✓ Hints, not spoilers — you still do the thinking

Compiled by Vaibhav Singh

★ If these notes help you, leave a star and come back for more ★



PART ONE

Java Programs

26 must-know Java programs — algorithm, working code, sample run, and complexity, explained clearly.



1. Reverse a Number

QUESTION

Write a program to reverse a given number.

ALGORITHM

1. Read the number from the user.
2. Initialize $rev = 0$.
3. Extract the last digit using $num \% 10$.
4. Add the digit to $rev \rightarrow rev = rev * 10 + digit$.
5. Remove the last digit $\rightarrow num = num / 10$.
6. Repeat steps 3 to 5 until num becomes 0.
7. Print rev .

CODE

```
import java.util.*;
public class ReverseNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int num = sc.nextInt();
        int rev = 0, r;
        while (num != 0) {
            r = num % 10;
            rev = rev * 10 + r;
            num = num / 10;
        }
        System.out.println("Reverse : " + rev);
    }
}
```

Input 12345

Output 54321

Time: $O(d)$ where d = number of digits

Space: $O(1)$

NOTE

- Any number obtained by reading the digits from right to left.
- Works for positive integers; handle sign separately for negative numbers.

2. Palindrome Number

QUESTION

Write a program to check whether a number is a palindrome or not.

ALGORITHM

1. Read the number from the user.
2. Store the number in another variable (originalNumber).
3. Reverse the number (same logic as Program 1).
4. Compare originalNumber and the reversed value.
5. If equal $\rightarrow$ palindrome, else $\rightarrow$ not palindrome.

CODE

```
import java.util.*;
public class PalindromeNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int num = sc.nextInt();
        int original = num;
        int rev = 0, r;
        while (num != 0) {
            r = num % 10;
            rev = rev * 10 + r;
            num = num / 10;
        }
        if (original == rev)
            System.out.println("Number is Palindrome");
        else
            System.out.println("Number is not Palindrome");
    }
}
```

Input 121

Output Number is Palindrome

Time: $O(d)$ where d = number of digits

Space: $O(1)$

NOTE

- A number that reads the same forwards and backwards is a palindrome.
- Examples: 121, 12321, 7, 1001.

3. Prime Number

QUESTION

Write a program to check whether a number is prime or not.

ALGORITHM

1. Read the number from the user.
2. If number $\leq 1 \rightarrow$ not prime.
3. Check divisibility from 2 up to $\sqrt{n}$.
4. If the number is divisible by any i , then it is not prime.
5. If not divisible by any $i \rightarrow$ prime.

CODE

```
import java.util.*;
public class PrimeNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int n = sc.nextInt();
        boolean isPrime = true;
        if (n <= 1) {
            isPrime = false;
        } else {
            for (int i = 2; i * i <= n; i++) {
                if (n % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }
        if (isPrime)
            System.out.println(n + " is a Prime");
        else
            System.out.println(n + " is not a Prime");
    }
}
```

Input 17

Output 17 is a Prime

Time: $O(\sqrt{n})$

Space: $O(1)$

NOTE

- A prime number has exactly two factors: 1 and itself.
- It is enough to test divisibility up to $\sqrt{n}$.
- Examples: 2, 3, 5, 7, 11, 13, 17.

4. Armstrong Number

QUESTION

Write a program to check whether a number is an Armstrong number or not.

ALGORITHM

1. Read the number from the user.
2. Count the total number of digits in the number.
3. For each digit, calculate $(\text{digit}^{\text{count}})$ and add it to sum.
4. If $\text{sum} == \text{original number} \rightarrow$ Armstrong, else $\rightarrow$ not Armstrong.

CODE

```
import java.util.*;
public class ArmstrongNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int n = sc.nextInt();
        int temp = n, sum = 0, count = 0;
        while (temp > 0) {
            temp = temp / 10;
            count++;
        }
        temp = n;
        while (temp > 0) {
            int digit = temp % 10;
            sum = sum + (int) Math.pow(digit, count);
            temp = temp / 10;
        }
        if (sum == n)
            System.out.println(n + " is Armstrong number");
        else
            System.out.println(n + " is not Armstrong number");
    }
}
```

Input 153

Output 153 is Armstrong number

Time: $O(d)$ where $d =$ number of digits

Space: $O(1)$

NOTE

- An Armstrong number equals the sum of its own digits, each raised to the power of the total digit count.
- Examples: 0, 1, 153, 370, 371, 407.

5. Odd or Even Number

QUESTION

Write a program to check whether a number is odd or even.

ALGORITHM

1. Read the number from the user.
2. If $\text{number} \% 2 == 0 \rightarrow \text{Even}$.
3. Else $\rightarrow \text{Odd}$.
4. Print the result.

CODE

```
import java.util.*;
public class OddEven {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int num = sc.nextInt();
        if (num % 2 == 0)
            System.out.println(num + " is Even");
        else
            System.out.println(num + " is Odd");
    }
}
```

Input 24

Output 24 is Even

Time: $O(1)$

Space: $O(1)$

NOTE

- If a number is exactly divisible by 2 it is Even, otherwise it is Odd.
- By convention, 0 is treated as Even.

6. Swap Two Numbers without a Third Variable

QUESTION

Write a program to swap two numbers without using a third variable.

ALGORITHM

1. Read two numbers a and b.
2. Use arithmetic operations to swap: $a = a + b$, $b = a - b$, $a = a - b$.
3. Print the swapped numbers.

CODE

```
import java.util.*;
public class SwapNumbers {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number : ");
        int a = sc.nextInt();
        System.out.print("Enter second number : ");
        int b = sc.nextInt();
        a = a + b; // Step 1
        b = a - b; // Step 2
        a = a - b; // Step 3
        System.out.println("After swapping :");
        System.out.println("a = " + a + ", b = " + b);
    }
}
```

Input a = 10, b = 25

Output a = 25, b = 10

Time: $O(1)$

Space: $O(1)$

NOTE

- This arithmetic approach only works reliably for integers, and $(a + b)$ may overflow for very large values.
- A bitwise XOR swap is an alternative.

7. Fibonacci Series

QUESTION

Write a program to print the Fibonacci series up to n terms.

ALGORITHM

1. Read the number of terms n from the user.
2. Initialize first = 0, second = 1.
3. Print the first two terms.
4. For i = 3 to n: next = first + second, print next, first = second, second = next.
5. Repeat step 4 until i > n.

CODE

```
import java.util.*;
public class FibonacciSeries {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of terms : ");
        int n = sc.nextInt();
        int first = 0, second = 1, next;
        System.out.print(first + " " + second + " ");
        for (int i = 3; i <= n; i++) {
            next = first + second;
            System.out.print(next + " ");
            first = second;
            second = next;
        }
    }
}
```

Input 7

Output 0 1 1 2 3 5 8

Time: O(n)

Space: O(1)

NOTE

- The Fibonacci series starts with 0 and 1.
- Every following term is the sum of the previous two terms.

8. Factorial of a Number

QUESTION

Write a program to find the factorial of a given number.

ALGORITHM

1. Read the number n from the user.
2. Initialize fact = 1.
3. For i = 1 to n: fact = fact * i.
4. Print fact.

CODE

```
import java.util.*;
public class FactorialNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number : ");
        int n = sc.nextInt();
        long fact = 1;
        for (int i = 1; i <= n; i++) {
            fact = fact * i;
        }
        System.out.println("Factorial of " + n + " is "
            + fact);
    }
}
```

Input 5

Output Factorial of 5 is 120

Time: O(n)

Space: O(1)

NOTE

- The factorial of 0 is defined as 1.
- Factorial values grow very quickly, so use long (or BigInteger) for larger inputs.

9. Sum of Digits of a Number

QUESTION

Write a program to calculate the sum of digits of a number.

ALGORITHM

1. Read the number from the user.
2. Initialize sum = 0.
3. Extract the last digit using $\text{num} \% 10$.
4. Add the digit to sum $\rightarrow \text{sum} = \text{sum} + \text{digit}$.
5. Remove the last digit $\rightarrow \text{num} = \text{num} / 10$.
6. Repeat steps 3 to 5 until num becomes 0.
7. Print sum.

CODE

```
import java.util.*;
public class SumOfDigits {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int num = sc.nextInt();
        int sum = 0;
        while (num > 0) {
            int digit = num % 10;
            sum = sum + digit;
            num = num / 10;
        }
        System.out.println("Sum of digits is : " +
sum);
    }
}
```

Input 12345

Output 15

Time: $O(d)$ where d = number of digits

Space: $O(1)$

NOTE

- Every digit is extracted with $\% 10$ and the number is shrunk with $/ 10$ until nothing remains.

10. Number of Digits in a Number

QUESTION

Write a program to count the number of digits in a given number.

ALGORITHM

1. Read the number from the user.
2. Initialize count = 0.
3. Divide the number by 10.
4. Increase count by 1.
5. Repeat steps 3 to 4 until num becomes 0.
6. Print count.

CODE

```
import java.util.*;
public class CountDigits {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int num = sc.nextInt();
        int count = 0;
        while (num > 0) {
            num = num / 10;
            count++;
        }
        System.out.println("Number of digits is : " +
count);
    }
}
```

Input 987654321

Output 9

Time: $O(d)$ where d = number of digits

Space: $O(1)$

NOTE

- Every division by 10 removes exactly one digit from the number, so counting the divisions counts the digits.

11. Reverse a String

QUESTION

Write a program to reverse a given string.

ALGORITHM

1. Read the string from the user.
2. Traverse the string from the end to the start.
3. Append each character to the result.
4. Print the reversed string.

CODE

```
import java.util.*;
public class ReverseString {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string : ");
        String str = sc.nextLine();
        String rev = "";
        for (int i = str.length() - 1; i >= 0; i--) {
            rev = rev + str.charAt(i);
        }
        System.out.println("Reversed String : " + rev);
    }
}
```

Input hello

Output olleh

Time: $O(n)$

Space: $O(n)$

NOTE

- The string is traversed once from the last character to the first, building the reversed copy.

12. Reverse Each Word of a String

QUESTION

Write a program to reverse each word of a given string.

ALGORITHM

1. Read the string from the user.
2. Split the string into words.
3. Reverse each word individually.
4. Print the words in their reversed form.

CODE

```
import java.util.*;
public class ReverseEachWord {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string : ");
        String str = sc.nextLine();
        String[] words = str.split(" ");
        String result = "";
        for (String word : words) {
            String rev = "";
            for (int i = word.length() - 1; i >= 0; i--) {
                rev = rev + word.charAt(i);
            }
            result = result + rev + " ";
        }
        System.out.println("Result : " + result.trim());
    }
}
```

Input Java is good

Output avaJ si doog

Time: $O(n)$

Space: $O(n)$

NOTE

- The sentence is split into words first, and each word is reversed on its own before being reassembled.

13. String Palindrome

QUESTION

Write a program to check whether a string is a palindrome or not.

ALGORITHM

1. Read the string from the user.
2. Compare characters from the start and end, moving towards the centre.
3. If all characters match → palindrome.
4. Else → not a palindrome.

CODE

```
import java.util.*;
public class StringPalindrome {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string : ");
        String str = sc.nextLine();
        String rev = "";
        for (int i = str.length() - 1; i >= 0; i--) {
            rev = rev + str.charAt(i);
        }
        if (str.equalsIgnoreCase(rev))
            System.out.println("String is Palindrome");
        else
            System.out.println("String is not Palindrome");
    }
}
```

Input madam

Output String is Palindrome

Time: $O(n)$

Space: $O(n)$

NOTE

- The string can also be compared without extra space using the two-pointer approach.
- `equalsIgnoreCase()` ignores letter case.

14. Check Two Strings are Anagrams

QUESTION

Write a program to check whether two strings are anagrams of each other.

ALGORITHM

1. Read two strings from the user.
2. If their lengths are different → not an anagram.
3. Sort the characters of both strings.
4. If the sorted strings are equal → anagram.
5. Else → not an anagram.

CODE

```
import java.util.*;
public class AnagramString {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first string : ");
        String str1 = sc.nextLine();
        System.out.print("Enter second string : ");
        String str2 = sc.nextLine();
        if (str1.length() != str2.length()) {
            System.out.println("Not Anagrams");
            return;
        }
        char[] ch1 = str1.toCharArray();
        char[] ch2 = str2.toCharArray();
        Arrays.sort(ch1);
        Arrays.sort(ch2);
        if (Arrays.equals(ch1, ch2))
            System.out.println("Strings are Anagrams");
        else
            System.out.println("Strings are not Anagrams");
    }
}
```

Input listen silent

Output Strings are Anagrams

Time: $O(n \log n)$

Space: $O(n)$

NOTE

- Two strings can only be anagrams if they have equal length.
- Sorting both strings and comparing them is the most common approach.

15. Count Occurrence of a Character in a String

QUESTION

Write a program to count the occurrence of a given character in a string.

ALGORITHM

1. Read the string and the character from the user.
2. Initialize count = 0.
3. Traverse the string.
4. If the character matches, increment count.
5. Print count.

CODE

```
import java.util.*;
public class CountChar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string : ");
        String str = sc.nextLine();
        System.out.print("Enter a character : ");
        char ch = sc.next().charAt(0);
        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            if (str.charAt(i) == ch) {
                count++;
            }
        }
        System.out.println("Occurrence of '" + ch + "'
is : " + count);
    }
}
```

Input Hello World , o

Output 2

Time: O(n)

Space: O(1)

NOTE

- The string is traversed once, comparing every character against the given character.

16. Count Number of Words in a String

QUESTION

Write a program to count the number of words in a string.

ALGORITHM

1. Read the string from the user.
2. Trim the leading and trailing spaces.
3. Split the string by one or more spaces.
4. Count the words.
5. Print count.

CODE

```
import java.util.*;
public class CountWords {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string : ");
        String str = sc.nextLine().trim();
        if (str.length() == 0) {
            System.out.println("No words");
            return;
        }
        String[] words = str.split("\\s+"); // one or
more spaces
        int count = words.length;
        System.out.println("Number of words : " +
count);
    }
}
```

Input Java is awesome

Output 3

Time: O(n)

Space: O(n)

NOTE

- trim() removes extra spaces at the start and end.
- split("\\s+") splits on one or more whitespace characters.

17. Find LCM of Two Numbers

QUESTION

Write a program to find the LCM of two numbers.

ALGORITHM

1. Read two numbers a and b.
2. Find the greater number.
3. Initialize lcm = greater.
4. While (true): if (lcm % a == 0 && lcm % b == 0) break; else lcm++.
5. Print lcm.

CODE

```
import java.util.*;
public class LCM {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number : ");
        int a = sc.nextInt();
        System.out.print("Enter second number : ");
        int b = sc.nextInt();
        int greater = (a > b) ? a : b;
        int lcm = greater;
        while (true) {
            if (lcm % a == 0 && lcm % b == 0)
                break;
            lcm++;
        }
        System.out.println("LCM is : " + lcm);
    }
}
```

Input 12 18

Output 36

Time: O(n)

Space: O(1)

NOTE

- LCM can also be calculated efficiently as $\text{LCM}(a, b) = (a * b) / \text{GCD}(a, b)$.

18. Find HCF (GCD) of Two Numbers

QUESTION

Write a program to find the HCF (GCD) of two numbers.

ALGORITHM

1. Read two numbers a and b.
2. While (b != 0): rem = a % b, a = b, b = rem.
3. Print a (which is the HCF).

CODE

```
import java.util.*;
public class HCF {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number : ");
        int a = sc.nextInt();
        System.out.print("Enter second number : ");
        int b = sc.nextInt();
        while (b != 0) {
            int rem = a % b;
            a = b;
            b = rem;
        }
        System.out.println("HCF (GCD) is : " + a);
    }
}
```

Input 48 18

Output 6

Time: O(log n)

Space: O(1)

NOTE

- This is the Euclidean Algorithm.
- It is very efficient even for large numbers.

19. Bubble Sort

QUESTION

Write a program to sort an array of numbers using Bubble Sort.

ALGORITHM

1. Read the array from the user.
2. For $i = 0$ to $n-2$, for $j = 0$ to $n-i-2$:
3. If $\text{arr}[j] > \text{arr}[j+1]$, swap $\text{arr}[j]$ and $\text{arr}[j+1]$.
4. After each outer pass the largest remaining element settles at the end.
5. Print the sorted array.

CODE

```
import java.util.*;
public class BubbleSort {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements : ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
        System.out.println("Sorted array : " + Arrays.toString(arr));
    }
}
```

Input 5\n5 1 4 2 8

Output [1, 2, 4, 5, 8]

Time: $O(n^2)$

Space: $O(1)$

NOTE

- Bubble Sort repeatedly swaps adjacent out-of-order elements.
- Simple to explain, but not efficient for large arrays.

20. Binary Search

QUESTION

Write a program to search an element in a sorted array using Binary Search.

ALGORITHM

1. Read a sorted array and the key to search.
2. Set $\text{low} = 0$ and $\text{high} = n - 1$.
3. While $\text{low} \leq \text{high}$: find $\text{mid} = (\text{low} + \text{high}) / 2$.
4. If $\text{arr}[\text{mid}] == \text{key}$, element found.
5. If $\text{arr}[\text{mid}] < \text{key}$, search the right half, else search the left half.

CODE

```
import java.util.*;
public class BinarySearch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements : ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        System.out.print("Enter element to search : ");
        int key = sc.nextInt();
        int low = 0, high = n - 1, result = -1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (arr[mid] == key) { result = mid; break; }
            else if (arr[mid] < key) low = mid + 1;
            else high = mid - 1;
        }
        if (result != -1)
            System.out.println("Element found at index : " + result);
        else
            System.out.println("Element not found");
    }
}
```

Input 6\n2 4 6 8 10 12, **Output** Element found at key = 10 index : 4

Time: $O(\log n)$

Space: $O(1)$

NOTE

- Binary Search only works on a sorted array.
- It halves the search space on every comparison, which is why it is much faster than a linear scan.

21. Second Largest Number in an Array

QUESTION

Write a program to find the second largest number in an array.

ALGORITHM

1. Read the array from the user.
2. Initialize largest and secondLargest to the smallest possible value.
3. Traverse the array; if element > largest, update secondLargest = largest, largest = element.
4. Else if element > secondLargest and element != largest, update secondLargest.
5. Print secondLargest.

CODE

```
import java.util.*;
public class SecondLargest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements : ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        int largest = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
        for (int num : arr) {
            if (num > largest) {
                second = largest;
                largest = num;
            } else if (num > second && num != largest) {
                second = num;
            }
        }
        System.out.println("Second largest number is : " + second);
    }
}
```

Input 5\n12 35 10 34 **Output** Second largest number is : 34

Time: O(n)

Space: O(1)

NOTE

- The array is traversed only once, tracking both the largest and second largest values together instead of sorting the whole array.

22. Remove Duplicate Elements from an Array

QUESTION

Write a program to remove duplicate elements from an array.

ALGORITHM

1. Read the array from the user.
2. Use a LinkedHashSet to preserve insertion order while removing duplicates.
3. Add every array element to the set.
4. Print the elements of the set.

CODE

```
import java.util.*;
public class RemoveDuplicates {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements : ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        LinkedHashSet<Integer> set = new LinkedHashSet<>();
        for (int num : arr) set.add(num);
        System.out.println("Array after removing duplicates : " + set);
    }
}
```

Input 7\n1 2 2 3 4 4 5 **Output** [1, 2, 3, 4, 5]

Time: O(n)

Space: O(n)

NOTE

- A LinkedHashSet automatically discards duplicate values while keeping the original insertion order intact.

23. Check Perfect Number

QUESTION

Write a program to check whether a number is a Perfect number or not.

ALGORITHM

1. Read the number from the user.
2. Initialize sum = 0.
3. For $i = 1$ to $n/2$, if $n \% i == 0$, add i to sum.
4. If $sum == n \rightarrow$ Perfect number, else $\rightarrow$ not Perfect.

CODE

```
import java.util.*;
public class PerfectNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int n = sc.nextInt();
        int sum = 0;
        for (int i = 1; i <= n / 2; i++) {
            if (n % i == 0) {
                sum = sum + i;
            }
        }
        if (sum == n)
            System.out.println(n + " is a Perfect
number");
        else
            System.out.println(n + " is not a Perfect
number");
    }
}
```

Input 28

Output 28 is a Perfect number

Time: $O(n)$

Space: $O(1)$

NOTE

- A Perfect number equals the sum of its proper divisors (excluding itself).
- Examples: 6, 28, 496, 8128.

24. Multiply Two Matrices

QUESTION

Write a program to multiply two matrices.

ALGORITHM

1. Read the dimensions and elements of matrix A and matrix B.
2. Check that the number of columns of A equals the number of rows of B.
3. For every result cell $[i][j]$, sum the products of the matching row of A and column of B.
4. Store the sum in the result matrix.
5. Print the result matrix.

CODE

```
import java.util.*;
public class MatrixMultiplication {
    public static void main(String[] args) {
        int[][] a = {{1, 2}, {3, 4}};
        int[][] b = {{5, 6}, {7, 8}};
        int rowsA = a.length, colsA = a[0].length,
        colsB = b[0].length;
        int[][] result = new int[rowsA][colsB];
        for (int i = 0; i < rowsA; i++) {
            for (int j = 0; j < colsB; j++) {
                for (int k = 0; k < colsA; k++) {
                    result[i][j] += a[i][k] * b[k][j];
                }
            }
        }
        System.out.println("Resultant Matrix :");
        for (int[] row : result) {
            System.out.println(Arrays.toString(row));
        }
    }
}
```

Input A = {{1,2},{3,4}}, B = {{5,6},{7,8}} **Output** [19, 22]
[43, 50]

Time: $O(n^3)$ for $n \times n$ matrices

Space: $O(n^2)$

NOTE

- Matrix multiplication needs three nested loops.
- The number of columns in the first matrix must equal the number of rows in the second.

25. Find Duplicate Characters in a String

QUESTION

Write a program to find duplicate characters and their count in a given string.

ALGORITHM

1. Read the string from the user.
2. Create a HashMap to store each character and its frequency.
3. Traverse the string and update the count for every character.
4. Print only the characters whose count is greater than 1.

CODE

```
import java.util.*;
public class DuplicateCharacters {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string : ");
        String str = sc.nextLine();
        HashMap<Character, Integer> map = new
        HashMap<>();
        for (char ch : str.toCharArray()) {
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }
        System.out.println("Duplicate characters :");
        for (Map.Entry<Character, Integer> entry :
        map.entrySet()) {
            if (entry.getValue() > 1) {
                System.out.println(entry.getKey() + " :
                " + entry.getValue());
            }
        }
    }
}
```

Input Better Butter **Output** t : 4, e : 3, r : 2, B : 2

Time: O(n)

Space: O(n)

NOTE

- A HashMap makes it easy to track how many times each character appears while scanning the string only once.

26. Check Harshad (Niven) Number

QUESTION

Write a program to check whether a number is a Harshad (Niven) number or not.

ALGORITHM

1. Read the number from the user.
2. Calculate the sum of its digits.
3. Check if the number is exactly divisible by the digit sum.
4. If divisible → Harshad number, else → not a Harshad number.

CODE

```
import java.util.*;
public class HarshadNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int n = sc.nextInt();
        int temp = n, sum = 0;
        while (temp > 0) {
            sum += temp % 10;
            temp /= 10;
        }
        if (n % sum == 0)
            System.out.println(n + " is a Harshad
            number");
        else
            System.out.println(n + " is not a Harshad
            number");
    }
}
```

Input 21

Output 21 is a Harshad number

Time: O(d) where d = number of digits

Space: O(1)

NOTE

- A Harshad (or Niven) number is divisible by the sum of its own digits.
- For example, 21 is a Harshad number because $21 / (2+1) = 7$.



PART TWO

DSA Question Bank

60 of the most commonly reported interview questions,
sorted Easy to Hard, with company tags and solving hints —
no spoilers.



Two Sum



Amazon, Google, Adobe, Microsoft

QUESTION

Given an array of integers and a target value, find the indices of the two numbers that add up to the target.

WHAT IT ASKS

You need to spot a pair of elements in the array whose sum equals the given target, and return their positions.

HINT

Checking every pair takes $O(n^2)$. Instead, walk through once and store each number's index in a hash map — at every step check if the 'complement' you need is already in the map.

Reverse a Linked List



Amazon, Microsoft, Adobe

QUESTION

Given the head of a singly linked list, reverse the list and return the new head.

WHAT IT ASKS

Flip the direction of every 'next' pointer so the last node becomes the first.

HINT

Keep three pointers — previous, current and next — and re-point current's next to previous as you move down the list in a single pass.

Valid Parentheses



Amazon, Google, Meta

QUESTION

Given a string containing only the brackets () { } [], determine whether the brackets are properly closed and nested.

WHAT IT ASKS

Check that every opening bracket has a matching closing bracket in the correct order.

HINT

A stack is perfect here: push opening brackets, and when a closing bracket appears, check whether it matches whatever is on top of the stack.

Maximum Subarray (Kadane's Algorithm)



Microsoft, Amazon, LinkedIn

QUESTION

Given an integer array, find the contiguous subarray with the largest sum and return that sum.

WHAT IT ASKS

Among all possible continuous slices of the array, find the one whose elements add up to the biggest total.

HINT

At every index decide whether to extend the running subarray or start fresh from the current element — track a 'current sum' and a 'best sum so far'.

Merge Two Sorted Lists



Amazon, Microsoft

QUESTION

Given the heads of two sorted linked lists, merge them into one sorted linked list.

WHAT IT ASKS

Combine both lists into a single list that stays sorted, reusing the existing nodes.

HINT

Use a dummy head node and repeatedly attach whichever of the two current nodes is smaller, then advance that list's pointer.

Best Time to Buy and Sell Stock



Amazon, Meta

QUESTION

Given an array where each element is a stock's price on a given day, find the maximum profit from one buy and one later sell.

WHAT IT ASKS

Pick a day to buy and a later day to sell so that the profit is as large as possible.

HINT

Track the lowest price seen so far while scanning left to right, and at each day compute the profit if you sold today.

Contains Duplicate



Amazon, Google

QUESTION

Given an integer array, determine whether any value appears at least twice.

WHAT IT ASKS

A simple check for whether the array has any repeated element.

HINT

A hash set lets you check 'have I seen this before?' in $O(1)$ time as you scan the array once.

Valid Anagram



Amazon, Bloomberg

QUESTION

Given two strings, determine whether the second is an anagram of the first.

WHAT IT ASKS

Check whether both strings use exactly the same letters with the same frequency, just rearranged.

HINT

Count character frequencies with a hash map (or a 26-length array for lowercase letters) and compare the two counts.

Move Zeroes

Meta, Amazon

QUESTION

Given an array, move all zeroes to the end while keeping the relative order of the non-zero elements, in place.

WHAT IT ASKS

Rearrange the array so non-zero numbers keep their order at the front and all zeroes settle at the back, without extra space.

HINT

Use a 'write pointer' that tracks where the next non-zero element should go, and swap as you scan through the array.

Climbing Stairs

Adobe, Amazon

QUESTION

You can climb 1 or 2 steps at a time. Given n stairs, count how many distinct ways there are to reach the top.

WHAT IT ASKS

Count the number of unique step combinations of 1s and 2s that add up to n .

HINT

The number of ways to reach step n equals the ways to reach step $n-1$ plus the ways to reach step $n-2$ — this is the Fibonacci sequence in disguise.

Single Number

Amazon, Airbnb

QUESTION

Given a non-empty array where every element appears twice except for one, find the single element that does not repeat.

WHAT IT ASKS

Identify the one value in the array that has no matching pair.

HINT

XOR-ing a number with itself gives zero, and XOR is order-independent — XOR every element together and the pairs will cancel out, leaving the unique number.

Missing Number

Amazon, Microsoft

QUESTION

Given an array containing n distinct numbers taken from the range 0 to n , find the one number missing from the range.

WHAT IT ASKS

Figure out which single number from $0..n$ is not present in the array.

HINT

The sum of $0..n$ has a simple formula. Subtract the actual array sum from that expected sum to get the missing number — or XOR the same way as Single Number.

Majority Element

Amazon, Adobe

QUESTION

Given an array of size n , find the element that appears more than $n/2$ times.

WHAT IT ASKS

Find the value that occurs in more than half of the array's positions.

HINT

The Boyer-Moore Voting technique keeps a running candidate and a counter — increment on a match, decrement otherwise, and swap candidates when the counter hits zero.

Palindrome Linked List

Amazon, Meta

QUESTION

Given the head of a singly linked list, determine whether it reads the same forwards and backwards.

WHAT IT ASKS

Check whether the sequence of values in the linked list is a palindrome.

HINT

Use a slow/fast pointer to find the middle, reverse the second half in place, then compare it against the first half node by node.

Symmetric Tree

Amazon, Bloomberg

QUESTION

Given the root of a binary tree, check whether it is a mirror of itself around its centre.

WHAT IT ASKS

Determine whether the left and right subtrees of the root are mirror images of each other.

HINT

Write a helper that compares two subtrees at a time, checking that their outer children match and their inner children match, recursively.

Maximum Depth of Binary Tree

Amazon, LinkedIn

QUESTION

Given the root of a binary tree, find its maximum depth (the number of nodes along the longest path from root to a leaf).

WHAT IT ASKS

Find how many levels deep the tree goes at its deepest point.

HINT

The depth of a tree is 1 plus the larger of the depths of its two subtrees — a simple recursive definition, or track depth explicitly with a BFS.

Intersection of Two Linked Lists

Amazon, Microsoft

QUESTION

Given the heads of two singly linked lists, find the node at which the two lists intersect, or return null if they don't.

WHAT IT ASKS

Find the exact node where two linked lists start sharing the same tail.

HINT

Walk both lists with two pointers; when one reaches the end, redirect it to the other list's head. The pointers meet exactly at the intersection after at most two passes.

First Unique Character in a String

Amazon, Bloomberg

QUESTION

Given a string, find the index of its first character that does not repeat, or -1 if none exists.

WHAT IT ASKS

Find the earliest character in the string that appears exactly once overall.

HINT

Count the frequency of every character in one pass with a hash map, then scan the string again and return the first character whose count is 1.

Implement strStr() (Substring Search)

Meta, Amazon

QUESTION

Given two strings, haystack and needle, return the index of the first occurrence of needle in haystack, or -1 if it isn't there.

WHAT IT ASKS

Find where one string first appears inside another.

HINT

A naive sliding comparison works in $O(n \cdot m)$; for something faster, look into the KMP algorithm's failure function to avoid re-checking matched characters.

Plus One

Google, Amazon

QUESTION

Given a non-empty array of digits representing a non-negative integer, add one to the number and return the resulting digit array.

WHAT IT ASKS

Simulate adding 1 to a large number that's stored digit by digit, handling any carry correctly.

HINT

Start from the last digit and add the carry; if a digit becomes 10, set it to 0 and carry 1 to the next position — if the carry survives past the first digit, prepend a new 1.

Product of Array Except Self



Amazon, Meta

QUESTION

Given an array, return a new array where each element is the product of all the other elements, without using division.

WHAT IT ASKS

For every index, multiply together all the numbers except the one at that index — without dividing.

HINT

Build a 'prefix product' array from the left and a 'suffix product' array from the right, then multiply the two for each index.

Merge Intervals



Meta, Google

QUESTION

Given a collection of intervals, merge all overlapping intervals and return the result.

WHAT IT ASKS

Combine any intervals that overlap or touch into single, larger intervals.

HINT

Sort the intervals by their start value first — once sorted, you only ever need to compare each interval with the last one you've merged.

Longest Substring Without Repeating Characters



Amazon, Bloomberg

QUESTION

Given a string, find the length of the longest substring that has no repeating characters.

WHAT IT ASKS

Find the longest 'window' of consecutive characters in the string where every character is unique.

HINT

Use a sliding window with two pointers and a hash set to track characters currently inside the window; shrink from the left whenever a duplicate appears.

3Sum



Amazon, Microsoft, Meta

QUESTION

Given an integer array, find all unique triplets of elements that sum to zero.

WHAT IT ASKS

Find every combination of three different array elements whose total is exactly zero, without duplicate triplets.

HINT

Sort the array first, fix one element, then use two pointers moving inward from both ends of the remaining array to find matching pairs.

Group Anagrams

Amazon, Uber

QUESTION

Given an array of strings, group the strings that are anagrams of each other.

WHAT IT ASKS

Bucket the words so each group contains only words made of the exact same letters.

HINT

Use a canonical form of each word (like its sorted letters) as a hash map key, and append the word to that key's list.

Rotate Array

Microsoft, Amazon

QUESTION

Given an array, rotate it to the right by k steps, in place.

WHAT IT ASKS

Shift every element k positions to the right, wrapping the overflow back to the start.

HINT

Reversing the whole array, then reversing the first k elements and the remaining $n-k$ elements separately, produces the rotation using $O(1)$ extra space.

Kth Largest Element in an Array

Amazon, Meta

QUESTION

Given an integer array and an integer k , find the k th largest element in sorted order.

WHAT IT ASKS

Without fully sorting, determine what value would sit in the k th position if the array were sorted in descending order.

HINT

A min-heap of size k (or the Quickselect partitioning technique) avoids a full $O(n \log n)$ sort.

Binary Tree Level Order Traversal

Amazon, Microsoft

QUESTION

Given the root of a binary tree, return the values of its nodes level by level, left to right.

WHAT IT ASKS

Group the node values by how far they are from the root, in the order they appear at each depth.

HINT

A queue-based breadth-first search naturally processes the tree one level at a time — track how many nodes belong to the current level before dequeuing them.

Course Schedule

Google, Amazon

QUESTION

Given a number of courses and a list of prerequisite pairs, determine whether it is possible to finish all the courses.

WHAT IT ASKS

Decide whether the prerequisite relationships form a cycle — if they do, the courses can never all be completed.

HINT

Model courses and prerequisites as a directed graph, then run a topological sort or a DFS that tracks nodes in the current recursion path to detect a cycle.

Number of Islands

Amazon, Google, Microsoft

QUESTION

Given a 2D grid of land and water cells, count the number of islands.

WHAT IT ASKS

Count how many separate connected groups of land cells exist, where connections are horizontal or vertical only.

HINT

When you find an unvisited land cell, run a DFS or BFS to 'sink' the whole connected island by marking its cells visited, counting how many times you start a new search.

Container With Most Water

Amazon, Google

QUESTION

Given an array of heights representing vertical lines, find two lines that together with the x-axis form the container holding the most water.

WHAT IT ASKS

Pick two lines so the rectangle of water between them (limited by the shorter line) is as large as possible.

HINT

Start with two pointers at both ends; the water is bounded by the shorter line, so move the pointer at the shorter line inward, since moving the taller one can never help.

Search in Rotated Sorted Array

Amazon, Microsoft

QUESTION

Given a sorted array that has been rotated at an unknown pivot, and a target value, find the target's index in $O(\log n)$.

WHAT IT ASKS

Locate a value inside an array that was sorted but then rotated, without scanning it linearly.

HINT

At each step of a binary search, one half of the array is guaranteed to still be normally sorted — figure out which half that is and check if the target lies within it.

Spiral Matrix

Amazon, Microsoft

QUESTION

Given an $m \times n$ matrix, return all its elements in spiral order.

WHAT IT ASKS

Read the matrix's values by walking around its outer boundary inward, layer by layer.

HINT

Track four boundaries — top, bottom, left, right — and shrink them inward after traversing each side of the current layer.

Validate Binary Search Tree

Amazon, Microsoft

QUESTION

Given the root of a binary tree, determine whether it is a valid binary search tree.

WHAT IT ASKS

Check that every node's value falls strictly between the allowed lower and upper bounds set by its ancestors.

HINT

Pass down a valid (min, max) range as you recurse; a node must fall inside that range, and it narrows the range for its own children.

Construct Binary Tree from Preorder and Inorder Traversal

Amazon, Microsoft

QUESTION

Given the preorder and inorder traversal arrays of a binary tree, reconstruct the tree.

WHAT IT ASKS

Rebuild the exact original tree structure from two of its traversal sequences.

HINT

The first element of preorder is always the root; find that value in inorder to know how many nodes belong to the left subtree, then recurse on both sides.

Coin Change

Amazon, Google

QUESTION

Given coin denominations and a target amount, find the fewest coins needed to make that amount (or -1 if impossible).

WHAT IT ASKS

Find the minimum number of coins that add up exactly to the target amount.

HINT

Build a DP array where $dp[i]$ is the minimum coins needed for amount i , using $dp[i] = \min(dp[i - \text{coin}] + 1)$ over all coins that fit.

Subsets

Amazon, Meta

QUESTION

Given an array of unique integers, return all possible subsets (the power set).

WHAT IT ASKS

List every possible combination of the array's elements, including the empty set and the full set.

HINT

At each element you have two choices — include it or skip it — backtracking through both choices naturally generates every subset.

Permutations

Amazon, Microsoft

QUESTION

Given an array of distinct integers, return all possible permutations.

WHAT IT ASKS

List every possible ordering of the given elements.

HINT

Backtrack by swapping elements into the current position one at a time, recursing on the remainder, then swapping back before trying the next option.

Top K Frequent Elements

Amazon, Meta

QUESTION

Given an integer array and an integer k, return the k most frequent elements.

WHAT IT ASKS

Find the k values that appear most often in the array.

HINT

Count frequencies with a hash map, then use a min-heap of size k (or bucket sort by frequency) to pull out the top k without a full sort.

Clone Graph

Amazon, Google

QUESTION

Given a reference to a node in a connected undirected graph, return a deep copy of the entire graph.

WHAT IT ASKS

Recreate the whole graph structure with brand-new node objects that mirror the original connections exactly.

HINT

Use a hash map from original nodes to their clones, and DFS or BFS through the graph, creating a clone the first time each node is visited and reusing it afterward.

Median of Two Sorted Arrays

Google, Microsoft

QUESTION

Given two sorted arrays, find the median of the combined data set.

WHAT IT ASKS

Find the middle value of both arrays merged together, without necessarily merging them, in better than linear time.

HINT

Binary search on the smaller array to find a partition point where the left halves of both arrays together hold exactly half of all the elements.

Trapping Rain Water

Amazon, Google, Goldman Sachs

QUESTION

Given an elevation map made of bars of different heights, compute how much rainwater it can trap after it rains.

WHAT IT ASKS

For each position, figure out how much water would sit above it, bounded by the tallest bars to its left and right.

HINT

The water trapped at any position depends on the minimum of the tallest bar to its left and the tallest bar to its right — a two-pointer sweep can track both in a single pass.

Word Ladder

Amazon, LinkedIn

QUESTION

Given a start word, an end word and a dictionary, find the length of the shortest transformation sequence that changes one letter at a time, with every intermediate word valid.

WHAT IT ASKS

Find the fewest single-letter swaps needed to turn the start word into the end word, only passing through dictionary words.

HINT

Treat each word as a graph node connected to words that differ by exactly one letter, then run a breadth-first search from the start word.

Longest Increasing Subsequence

Google, Microsoft

QUESTION

Given an integer array, find the length of the longest strictly increasing subsequence.

WHAT IT ASKS

Find the longest sequence of elements (not necessarily contiguous) that keeps increasing in value while preserving their original order.

HINT

A DP array where $dp[i]$ is the LIS ending at index i gives an $O(n^2)$ solution; for $O(n \log n)$, maintain a growing 'tails' array and binary search for where each new element belongs.

Serialize and Deserialize Binary Tree



Amazon, Meta

QUESTION

Design an algorithm to convert a binary tree into a string, and convert that string back into the original tree.

WHAT IT ASKS

Come up with an encoding scheme that captures the full shape and values of the tree so it can be rebuilt exactly.

HINT

A pre-order traversal that explicitly records 'null' markers for missing children lets you reconstruct the tree recursively from that same sequence.

LRU Cache



Amazon, Microsoft, Google

QUESTION

Design a data structure that supports get and put operations in $O(1)$, evicting the least recently used item whenever capacity is exceeded.

WHAT IT ASKS

Build a fixed-size cache that always knows which item was used longest ago, so it can be the first one removed when space runs out.

HINT

Combine a hash map (for $O(1)$ lookup) with a doubly linked list (to track usage order), moving a node to the front on every access.

N-Queens



Microsoft, Amazon

QUESTION

Place n queens on an $n \times n$ chessboard so that no two queens attack each other, and return all distinct solutions.

WHAT IT ASKS

Find every arrangement of n queens where no two share a row, column, or diagonal.

HINT

Place queens row by row using backtracking, keeping track of which columns and diagonals are already occupied so invalid placements can be skipped early.

Edit Distance



Google, Amazon

QUESTION

Given two strings, find the minimum number of insert, delete, or replace operations needed to convert one into the other.

WHAT IT ASKS

Find the cheapest sequence of single-character edits that transforms the first word into the second.

HINT

Build a 2D DP table where $dp[i][j]$ is the edit distance between the first i and j characters of each string, branching on whether the current characters match.

Merge k Sorted Lists

Amazon, Google, Meta

QUESTION

Given k sorted linked lists, merge them all into one sorted linked list.

WHAT IT ASKS

Combine all k lists into a single, fully sorted list.

HINT

A min-heap holding the current front node of each list lets you always pull the smallest available node in $O(\log k)$ time.

Word Break

Google, Meta

QUESTION

Given a string and a dictionary of words, determine whether the string can be segmented into a space-separated sequence of dictionary words.

WHAT IT ASKS

Check whether the string can be fully split into pieces that all exist in the given word list.

HINT

Use DP where $dp[i]$ is true if the prefix ending at i can be segmented — check every $j < i$ where $dp[j]$ is true and the substring from j to i is a dictionary word.

Regular Expression Matching

Google, Meta

QUESTION

Given a string and a pattern with support for '.' and '*', implement regular expression matching that covers the entire input string.

WHAT IT ASKS

Determine whether the whole string matches the pattern, where '.' matches any single character and '*' matches zero or more of the preceding character.

HINT

Build a 2D DP table where $dp[i][j]$ tracks whether the first i characters of the string match the first j characters of the pattern, handling '*' as a special two-case transition.

Sliding Window Maximum

Amazon, Google

QUESTION

Given an array and a window size k , return the maximum value in every sliding window of size k as it moves across the array.

WHAT IT ASKS

For every position of a size- k window sliding left to right, report the largest value currently inside it.

HINT

Use a deque that stores indices in decreasing order of value — pop smaller elements from the back before pushing a new one, and drop indices from the front once they fall outside the window.

Longest Valid Parentheses

Amazon, Google

QUESTION

Given a string containing only '(' and ')', find the length of the longest valid (well-formed) parentheses substring.

WHAT IT ASKS

Find the longest contiguous stretch of the string that forms correctly matched parentheses.

HINT

A stack storing indices works well here — push indices of unmatched characters, and measure the gap between the current index and the new stack top whenever a match closes.

Largest Rectangle in Histogram

Amazon, Google

QUESTION

Given an array representing the heights of histogram bars of width 1, find the area of the largest rectangle that fits within the histogram.

WHAT IT ASKS

Find the biggest rectangular area that can be formed using consecutive bars, where the rectangle's height is limited by the shortest bar it spans.

HINT

Use a monotonic increasing stack of bar indices — when a shorter bar appears, pop and compute the area for each taller bar that can no longer extend rightward.

Minimum Window Substring

Amazon, Meta

QUESTION

Given two strings s and t, find the smallest substring of s that contains every character of t (including duplicates).

WHAT IT ASKS

Find the shortest window in s that has enough of every character required by t.

HINT

A sliding window with two pointers and a character-count map works well: expand the right edge until the window is valid, then shrink from the left to find the minimum valid window.

Alien Dictionary

Amazon, Google

QUESTION

Given a list of words sorted according to the rules of an unknown alien alphabet, determine a valid ordering of the alphabet's letters.

WHAT IT ASKS

Figure out the character ordering rules implied by a sorted word list from an unfamiliar alphabet.

HINT

Compare each pair of adjacent words to find the first differing letter, which gives you a 'comes before' edge — then topologically sort the resulting letter graph.

Word Search II

Amazon, Google

QUESTION

Given a 2D board of letters and a list of words, find all words from the list that can be formed by tracing adjacent cells on the board.

WHAT IT ASKS

Find every word from the list that can be spelled out by moving between neighbouring letters on the grid without reusing a cell.

HINT

Build a Trie from the word list first, then DFS from every cell, following the Trie so you can stop early whenever no word starts with the letters seen so far.

Best Time to Buy and Sell Stock III

Amazon, Microsoft

QUESTION

Given stock prices for each day, find the maximum profit achievable with at most two buy-sell transactions.

WHAT IT ASKS

Maximise total profit when you're allowed to buy and sell up to two separate times, without overlapping transactions.

HINT

Track four running values as you scan once: the best profit after the first buy, after the first sell, after the second buy, and after the second sell — each depends only on the previous one.

Basic Calculator

Google, Meta

QUESTION

Implement a basic calculator that evaluates a string expression containing integers, '+', '-', parentheses, and spaces.

WHAT IT ASKS

Parse and correctly evaluate an arithmetic expression string, respecting nested parentheses.

HINT

Use a stack to remember the running result and sign whenever you enter a new parenthesis, then restore and combine them when that parenthesis closes.

Maximal Rectangle

Amazon, Google

QUESTION

Given a 2D binary matrix filled with 0s and 1s, find the area of the largest rectangle containing only 1s.

WHAT IT ASKS

Find the biggest all-1s rectangle hidden inside the matrix.

HINT

Treat each row as the base of a histogram, adding up consecutive 1s column-wise from the rows above, then reuse the Largest Rectangle in Histogram technique on every row.